



Entorno de prácticas. Hola mundo. Práctica 0

Raúl Fernández
Lía García
David Alejo



- Vamos a realizar las prácticas en C++
- Usaremos la librería PhysX de Nvidia
- Usamos la versión v4.1 para mayor compatibilidad que la v5.X.
- En nuestro caso la V5.X no añade ninguna funcionalidad nueva.
- Empezaremos haciendo toda la simulación nosotros mismos
- Usando alguna clase de PhysX
 - Así no cambiamos de entorno cuando empecemos a usarlo para simular



- Al usar ciertos aspectos de PhysX es necesario usar sus librerías
- En los enunciados de las prácticas se indicará cómo usar funcionalidades necesarias
- Para más información:
 - [PhysX User's Guide](#)
 - [OpenGL Utility Toolkit \(GLUT\)](#)



- <https://github.com/UCM-237/SimulacionFisicaVideojuegos>
 - Contiene todo lo necesario para comenzar a programar



- ☐ Las entregas se realizarán mediante git
- ☐ Cada estudiante tendrá un fork al repositorio de la asignatura donde irá subiendo el código de las prácticas
- ☐ La entrega se realizará etiquetando la versión con una etiqueta. El número de versión será:
 - ☐ V0.P.A (P número de la práctica y A apartado)
 - ☐ Por ejemplo, el apartado 3 de la práctica 1 se entregará como V0.1.3
- ☐ Las prácticas y el proyecto intermedio/final son **INDIVIDUALES**.
- ☐ Durante la corrección y supervisión de las prácticas y el proyecto os podemos pedir que cambiéis código o que nos respondáis a cuestiones de física o programación relacionadas con la práctica.

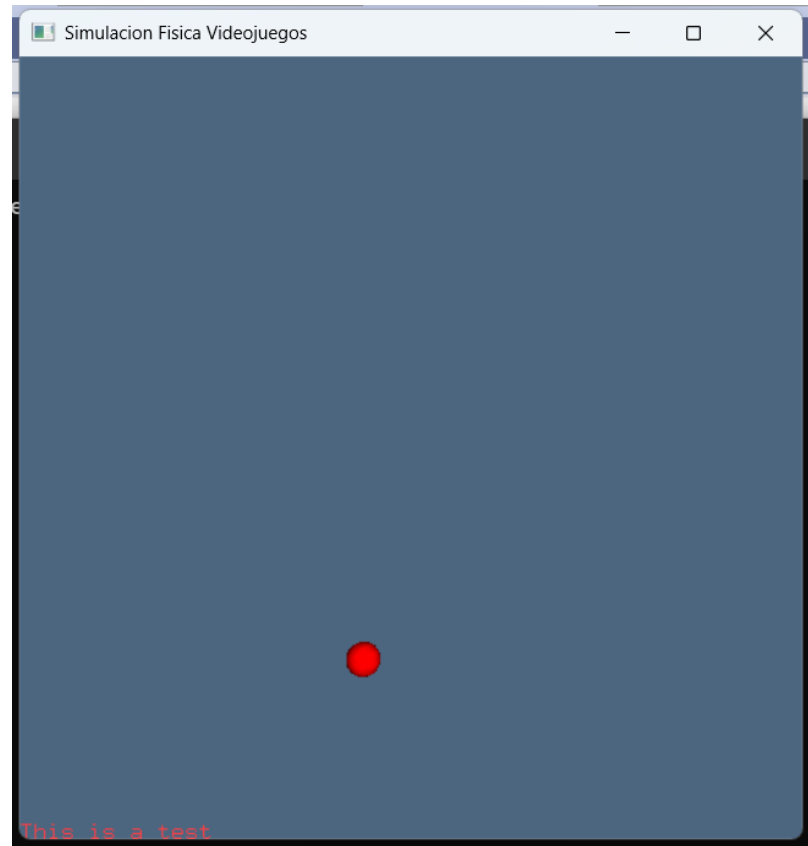


Primera parte: Hola Mundo



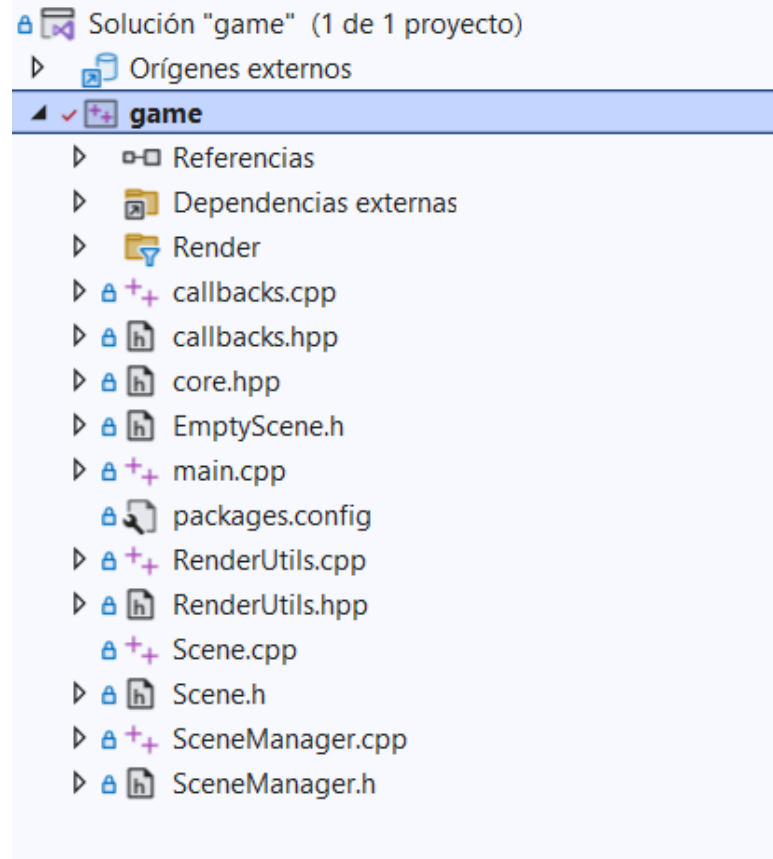
Actividad 1: Entorno de prácticas

Hola Mundo





Actividad 1: Entorno de prácticas





Actividad 1: Entorno de prácticas

- ❑ El main.cpp es el punto de partida
- ❑ Tenemos cuatro funciones básicas:

```
> void initPhysics(bool interactive) { ... }  
  
✓ // Function to configure what happens in each step of physics  
| // interactive: true if the game is rendering, false if it offline  
> void stepPhysics(bool interactive, double t) { ... }  
  
// Add custom code to the beginning of the function  
> void cleanupPhysics(bool interactive) { ... }  
  
// Function called when a key is pressed  
> void keyPress(unsigned char key, const PxTransform& camera) { ... }
```



```
> void initPhysics(bool interactive) { ... }
```

- ☐ Se llama automáticamente al principio del juego
- ☐ Se utiliza para inicializar todos los elementos necesarios para la simulación
- ☐ Creación de instancias básicas de PhysX
- ☐ Este es el lugar para el código de inicialización



```
> void stepPhysics(bool interactive, double t) { ... }
```

- ☐ Función que se llamará para cada paso de simulación
- ☐ Será necesario completarla si necesitamos que nuestra práctica haga algo en cada Frame (actualice posiciones, por ejemplo)
- ☐ Recibe el tiempo transcurrido desde la última llamada



```
> void keyPress(unsigned char key, const PxTransform& camera) { ... }
```

- ❑ Además tenemos esta función para interactuar con el teclado, que podemos completar.
- ❑ Para evitar conflictos con la práctica 5 se recomienda que todas las teclas estén asignadas a letras, y no usar números.



```
> void cleanupPhysics(bool interactive) { ... }
```

- ☐ Llamada automáticamente al terminar la simulación al cerrar el juego
- ☐ Limpiar todo aquello que hayamos creado



- ❑ Todas las funciones y clases de PhysX usan un namespace physx
- ❑ using namespace physx
- ❑ Algunas configuraciones básicas en core.hpp



Parte 2: pintar



Geometría

- `PxSphereGeometry`
- `PxBoxGeometry`
- `PxCapsuleGeometry`
- `PxPlaneGeometry`



Crear una forma a partir de esa geometría

- `CreateShape`
 - `PxGeometry`
 - `PxTransform`
 - `Color`



Crear un item para renderizar

- `RenderItem`



- ☐ El sistema de pintado está ya programado
- ☐ Basado en OpenGL y glut
- ☐ El centro es una clase llamada RenderItem (Definida en **RenderUtils**)
- ☐ Realiza toda la funcionalidad necesaria
- ☐ Podéis extenderla con lo que necesitéis
- ☐ Pero no es imprescindible tocarla
- ☐ Si falta algo para hacer la práctica hablad con un profesor



- ❑ El sistema de pintado está encapsulado
- ❑ RenderItem es la clase que se utiliza para registrar algo para pintar
- ❑ Contiene
 - ❑ Shape con la geometría a pintar
 - ❑ Transform para saber su posición y orientación
 - ❑ Color



- ❑ `RenderItem(physx::PxShape* _shape, const physx::PxTransform* _trans, Vector3 _color)`
- ❑ El primer atributo de RenderItem es **shape** es un puntero a instancia shape de PhysX
- ❑ Usaremos geometrías de Physx. Todas derivan de una clase general PxGeometry
 - ❑ PxSphereGeometry
 - ❑ PxBBoxGeometry
 - ❑ PxCapsuleGeometry



- ❑ `RenderItem(physx::PxShape* _shape, const physx::PxTransform* _trans, Vector3 _color)`
- ❑ El atributo **transform** es un puntero a una instancia Transform
- ❑ Es una clase de PhysX
- ❑ Encapsula una posición, rotación
- ❑ Puntero ya que cuando algo se mueva lo tendremos que ir actualizando
- ❑ El código del juego guarda una instancia a Transform
- ❑ RenderItem guarda el puntero a esa instancia
- ❑ Siempre se pinta en la posición actualizada

```
PxTransform(const PxVec3& position) : q(PxIdentity), p(position)
```

```
PxTransform(const PxQuat& orientation) : q(orientation), p(0)
```

```
PxTransform(const PxVec3& p0, const PxQuat& q0) : q(q0), p(p0)
```



- ❑ *RenderItem(physx::PxShape* _shape, **const physx::PxTransform*** _trans, Vector3 _color)*
- ❑ El último atributo es **color**
- ❑ Vector4 {R,G,B, alpha}
- ❑ Formato RGB con valores entre 0 y 1
- ❑ Alpha transparencia.
 - ❑ 1 objeto sólido
 - ❑ 0 objeto transparente



- ❑ ¿Como borrar nuestros elementos?
- ❑ PhysX lleva a cabo un conteo de referencia para la eliminación de sus elementos. **Cuando un elemento no es referenciado lo elimina.** No usamos delete () usamos **release()**.
- ❑ Los RenderItem los borramos con su método release(). Este método también lleva a cabo un conteo de referencias y a su vez llama al método release() de Shape.
- ❑ **CUIDADO:** RenderItem usa Shape que es parte de PhysX. Si Shape tiene más referencias que el RenderItem (ejemplo variable externa) no se borrará y quedará en memoria.



Parte 3: clase Vector3D



- ❑ PhysX tiene implementada una clase de Vectores 3D
- ❑ PERO por motivos didácticos vamos a crear la nuestra de manera que podamos hacer todas las operaciones que necesitemos usando vectores en el espacio



Reto A

